

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

背包 DP：从 0/1 到分组

Knapsack Dynamic Programming: From 0/1 to Grouped

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 背包问题概述
2. 0/1 背包
3. 完全背包
4. 多重背包
5. 分组背包
6. 经典例题

目录

⑦ 总结

目录

1. 背包问题概述
2. 0/1 背包
3. 完全背包
4. 多重背包
5. 分组背包
6. 经典例题

目录

7 总结

什么是背包问题

背包问题：给定一组物品（每件有重量和价值）和一个固定容量的背包，要求在容量限制下选出若干物品，使总价值最大。

三个要素

- **物品**：每件物品有重量 $w[i]$ 和价值 $v[i]$
- **容量**：背包能装下的总重量上限 W
- **目标**：在总重量不超过 W 的前提下，价值最大化

背包 = 有限容量下的最优选择问题

背包问题的分类

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

类型	每件物品能取几次	一维容量遍历方向	转移依赖
0/1 背包	最多 1 次	倒序	上一行
完全背包	无限次	正序	本行
多重背包	有限 $c[i]$ 次	拆成 0/1	上一行
分组背包	每组选一个	组内倒序	上一组

四类背包本质相同：在容量约束下做选择，只是物品的可用次数不同

核心思想：状态定义 + 转移

背包 DP 的本质：用一个状态描述“已经考虑过的物品 + 已用的容量”，用状态之间的转移表示“选或不选”。

状态一般形式

- 二维：$dp[i][j]$ 表示前 i 件物品、容量为 j 时的最优解
- 一维：$dp[j]$ 表示容量为 j 时的最优解（滚动掉物品维）

转移一般形式

- 不选第 i 件：继承上一状态
- 选第 i 件：腾出 $w[i]$ 的空间，加上 $v[i]$

所有背包问题都在回答同一个问题：第 i 件物品，选还是不选？

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题

目录

7 总结

例题一：0/1 背包

题目：有 n 件物品和一个容量为 W 的背包。第 i 件物品的重量是 $w[i]$，价值是 $v[i]$。每件物品只能取一次，求在总重量不超过 W 的前提下，能获得的最大总价值。

示例

- $n = 3$，$W = 5$
- 重量 $w = [2,\ 3,\ 4]$，价值 $v = [3,\ 4,\ 5]$
- 最优：选第 1 件和第 2 件，总重量 5，总价值 7

0/1 背包：解法

四步法

- 状态：$dp[i][j]$ 表示前 i 件物品、容量为 j 时的最大价值
- 转移：对第 i 件物品，选或不选，取较大者
- 方程：$dp[i][j] = \max(dp[i-1][j],\ dp[i-1][j-w[i]] + v[i])$
- 边界：$dp[0][j] = 0$，一件不选时价值为 0

两种选择

- 不选第 i 件：价值等于前 $i-1$ 件、容量 j 的最优解
- 选第 i 件：先腾出 $w[i]$ 的空间，再加上第 i 件的价值

每件物品只有两种命运：进背包，或留在原地

0/1 背包：二维代码


```cpp
// dp[i][j] 表示前 i 件物品、容量 j 时的最大价值
int knapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i-1][j];                  // 不选第 i 件
            if (j >= w[i-1]) {
                // 选第 i 件，容量腾出 w[i-1] *
                dp[i][j] = max(dp[i][j],
                               dp[i-1][j - w[i-1]] + v[i-1]);
            }
        }
    }
    return dp[n][W];
}
```

二维写法最直观，每一行只依赖上一行，是理解背包的起点。

0/1 背包：一维优化

观察：$dp[i][\cdot]$ 只依赖 $dp[i-1][\cdot]$，物品维可以滚动掉。

一维状态

- $dp[j]$ 表示容量为 j 时的最大价值
- **转移**：$dp[j] = \max(dp[j],\ dp[j-w[i]] + v[i])$

遍历顺序：**容量必须倒序**

- 倒序 j 从 W 到 $w[i]$：$dp[j-w[i]]$ 还是“上一轮”的值
- 正序 j 从 $w[i]$ 到 W：$dp[j-w[i]]$ 已被本轮更新，会重复取

倒序是为了保证每件物品只被取一次

0/1 背包：一维代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// dp[j] 表示容量为 j 时的最大价值
int knapsackOpt(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 容量从大到小遍历，保证每件物品只被取一次 *
        for (int j = W; j >= w[i]; j--) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    return dp[W];
}
```

一维 + 倒序，是 0/1 背包的标准写法，务必写熟

为什么倒序？——一维数组的视角

倒序（正确）

- j 从 W 递减到 $w[i]$
- 计算 $dp[j]$ 时，$dp[j-w[i]]$ 尚未被本轮更新
- 读到的是“前 $i-1$ 件”的最优值
- 每件物品只被用一次

正序（错误）

- j 从 $w[i]$ 递增到 W
- 计算 $dp[j]$ 时，$dp[j-w[i]]$ 已被本轮更新
- 读到的是“前 i 件”的最优值
- 相当于第 i 件可以反复取

倒序 $=0/1$ 背包；正序 $=$ 完全背包。这个差别只有一行，但后果完全不同

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题

目录

7 总结

例题二：完全背包

题目：有 n 种物品和一个容量为 W 的背包。第 i 种物品的重量是 $w[i]$，价值是 $v[i]$，每种物品可以取**无限次**，求在总重量不超过 W 的前提下，能获得的最大总价值。

示例

- $n = 3$，$W = 10$
- 重量 $w = [3,\ 4,\ 5]$，价值 $v = [4,\ 5,\ 6]$
- 最优：取 2 个第 1 种 + 1 个第 2 种，总重量 10，总价值 13

完全背包：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[j]$ 表示容量为 j 时的最大价值
- 转移：$dp[j] = \max(dp[j],\ dp[j - w[i]] + v[i])$
- 边界：$dp[0] = 0$，其余初始化为 0
- 顺序：容量**正序**遍历，因为同一物品可以反复取

为什么正序就对？

- 计算 $dp[j]$ 时，$dp[j - w[i]]$ 已经在本轮被更新过
- 也就是说，$dp[j - w[i]]$ 里可能已经含了一个第 i 种物品
- 再加一个，正是“同一物品反复取”的效果

完全背包与 0/1 背包的唯一区别：容量遍历方向

完全背包：代码


```cpp
// dp[j] 表示容量为 j 时的最大价值
int completeKnapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 容量从小到大遍历，允许同一物品重复取 *
        for (int j = w[i]; j <= W; j++) {
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    return dp[W];
}
```

与 0/1 背包相比，只把倒序改成正序，代码结构完全一样

0/1 与完全：对比记忆

对比项	0/1 背包	完全背包
物品可用次数	每件最多 1 次	每种无限次
容量遍历方向	倒序	正序
内层循环写法	j 从 W 到 $w[i]$	j 从 $w[i]$ 到 W
转移读到的状态	$dp[i-1][\cdot]$	$dp[i][\cdot]$
典型问题	采药、装箱	零钱兑换、完全背包

记住一句话：倒序 0/1，正序完全；差别只有一行

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题

目录

⑦ 总结

例题三：多重背包

题目：有 n 种物品和一个容量为 W 的背包。第 i 种物品的重量是 $w[i]$，价值是 $v[i]$，最多可以取 $c[i]$ 件，求在总重量不超过 W 的前提下，能获得的最大总价值。

示例

- $n = 2$，$W = 8$
- 物品 1：$w = 2$，$v = 3$，$c = 3$
- 物品 2：$w = 3$，$v = 4$，$c = 2$
- 最优：取 1 个物品 $1 + 2$ 个物品 2，总重量 8，总价值 11

多重背包：朴素解法

思路一：拆成 0/1 背包

- 第 i 种物品有 $c[i]$ 件，把它看成 $c[i]$ 件独立的物品
- 每件只取一次，就用 $0/1$ 背包的写法处理
- 总物品数变成 $\sum c[i]$，复杂度 $O(W \sum c[i])$

问题

- 如果 $c[i]$ 很大（比如 10^9），逐个拆分直接超时
- 需要更聪明的拆分方式：二进制拆分

朴素解法能解决小规模，大规模需要二进制优化

多重背包：二进制优化

核心思想：把 $c[i]$ 件拆成若干个“包”，每个包里件数是 $1, 2, 4, 8, \ldots$。

为什么可以

- 任意 0 到 $c[i]$ 之间的整数，都能由这些二进制组合出来
- 例：$c = 13 = 1 + 2 + 4 + 6$，$0 \sim 13$ 都能凑
- 拆分后每包视为一件独立物品，用 $0/1$ 背包处理

复杂度

- 原来：$\sum c[i]$ 件物品
- 现在：$\sum \log c[i]$ 件物品
- 从 $O(W \sum c[i])$ 降到 $O(W \sum \log c[i])$

用二进制的语言，把“取几件”编码成“取哪几包”

多重背包：二进制优化代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// 把 cnt[i] 件物品 i 拆成若干包，转化为 0/1 背包
int multiKnapsack(int n, int W, vector<int>& w,
                  vector<int>& v, vector<int>& cnt) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int c = cnt[i];
        // 按 1, 2, 4, ... 拆分 *
        for (int k = 1; k <= c; k <<= 1) {
            c -= k;
            int weight = k * w[i];
            int value  = k * v[i];
            for (int j = W; j >= weight; j--) {   // 0/1 写法，倒序 *
                dp[j] = max(dp[j], dp[j - weight] + value);
            }
        }
        if (c > 0) {                              // 剩下的部分单独成包
            int weight = c * w[i];
            int value  = c * v[i];
```

多重背包：二进制优化代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19              for (int j = W; j >= weight; j--) {
20                  dp[j] = max(dp[j], dp[j - weight] + value);
21              }
22          }
23      }
24      return dp[W];
25  }
```

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题



目录

7 总结

例题四：分组背包

题目：有 n 组物品和一个容量为 W 的背包。第 i 组里有若干物品，每组**最多只能选一件**，求在总重量不超过 W 的前提下，能获得的最大总价值。

示例

- 2 组，$W = 6$
- 第 1 组：A$(w = 2, v = 3)$，B$(w = 3, v = 4)$
- 第 2 组：C$(w = 2, v = 4)$，D$(w = 4, v = 6)$
- 最优：选 A + D，总重量 6，总价值 9

分组背包：解法

四步法

- 状态：$dp[j]$ 表示容量为 j 时的最大价值
- 转移：外层枚举每组，中层枚举容量，内层枚举组内物品
- 方程：$dp[j] = \max(dp[j],\ dp[j - w_k] + v_k)$，$k$ 为该组物品
- 边界：$dp[0] = 0$，其余初始化为 0

三层循环的顺序

- 第一层：枚举每一组
- 第二层：容量从 W 倒序到 0（保证每组只选一次）
- 第三层：枚举组内物品，看选哪个最优

分组背包 = 每组内部做一次 0/1 背包

分组背包：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// groups[i] 为第 i 组物品列表，每个元素是 {重量，价值}
int groupKnapsack(int n, int W,
                  vector<vector<pair<int,int>>>& groups) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        // 容量倒序，保证每组最多选一件 *
        for (int j = W; j >= 0; j--) {
            for (auto& item : groups[i]) {
                int wt = item.first, val = item.second;
                if (j >= wt)
                    dp[j] = max(dp[j], dp[j - wt] + val);
            }
        }
    }
    return dp[W];
}
```

分组背包：代码

二层倒序是关键：如果写正序，同一组就可能被选多次

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题

目录

7 总结

例题五：分割等和子集

题目：给定一个只含正整数的数组 $nums$，判断能否将它分割成两个子集，使得两个子集的元素和相等。

示例

- $nums = [1,\ 5,\ 11,\ 5]$，**答案为** true
- **一种分法**：$[1,\ 5,\ 5]$ **与** $[11]$
- $nums = [1,\ 2,\ 3,\ 5]$，**答案为** false

分割等和子集：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

转化为 0/1 背包

- 先求和，若总和为奇数，直接返回 false
- 目标：从数组里选一些数，使它们的和等于 $target = \text{sum}/2$
- 每个数最多选一次，是典型的 0/1 背包

四步法

- 状态：$dp[j]$ 表示能否凑出和 j，布尔型
- 转移：$dp[j] = dp[j]$ 或 $dp[j - nums[i]]$
- 边界：$dp[0] = \text{true}$
- 答案：$dp[target]$

“可行性”背包：状态是布尔值，转移用“或”

分割等和子集：代码


```cpp
// dp[j] 表示能否凑出和为 j
bool canPartition(vector<int>& nums) {
    int sum = 0;
    for (int x : nums) sum += x;
    if (sum % 2 != 0) return false;          // 和为奇数必不可能

    int target = sum / 2;
    vector<bool> dp(target + 1, false);
    dp[0] = true;                            // 凑出 0 一定可以 *

    for (int x : nums) {
        // 容量倒序，每个数只取一次 *
        for (int j = target; j >= x; j--) {
            dp[j] = dp[j] || dp[j - x];
        }
    }
    return dp[target];
}
```

分割等和子集：代码

0/1 背包的“可行性”变体：把“最大价值”换成“能否凑出”

例题六：零钱兑换

题目：给定不同面额的硬币 $coins$ 和一个总金额 $amount$，每种硬币可以使用**无限次**，求凑出金额 $amount$ 所需的**最少硬币数**。如果凑不出，返回 -1。

示例

- $coins = [1,\ 2,\ 5]$，$amount = 11$，**答案为** 3 $(5 + 5 + 1)$
- $coins = [2]$，$amount = 3$，**答案为** -1

零钱兑换：解法

完全背包的“最少数量”变体

- 状态：$dp[j]$ 表示凑出金额 j 所需的最少硬币数
- 转移：$dp[j] = \min(dp[j],\ dp[j-x]+1)$，$x$ 为硬币面额
- 边界：$dp[0] = 0$，其余初始化为一个极大值
- 顺序：金额**正序**遍历，因为硬币可以反复用

与 0/1 背包的对比

- 目标从“最大化价值”变为“最小化数量”
- 转移从 $\max$ 变为 $\min$
- 容量从倒序变为正序（完全背包）

背包问题的“目标函数”可以灵活变：最大价值、最少数量、方案数

零钱兑换：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// dp[j] 表示凑出金额 j 所需的最少硬币数
int coinChange(vector<int>& coins, int amount) {
    const int INF = 1e9;
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;                                  // 凑 0 元不需要硬币 *

    for (int x : coins) {
        // 金额正序，硬币可重复使用 *
        for (int j = x; j <= amount; j++) {
            dp[j] = min(dp[j], dp[j - x] + 1);
        }
    }
    return dp[amount] >= INF ? -1 : dp[amount];
}
```

正序 + min：完全背包的“最少数量”写法

例题七：目标和

题目：给定数组 $nums$ 和整数 $target$，给数组里每个数前面加 $+$ 或 $-$，求使最终和为 $target$ 的不同方案数。

示例

- $nums = [1,\ 1,\ 1,\ 1,\ 1]$，$target = 3$，答案为 5
- **一种分法**：$+1+1+1+1-1=3$
- **另一种**：$+1-1+1+1+1=3$

目标和：解法

转化为 0/1 背包（计数型）

- 设正号部分的和为 P，负号部分的和为 N
- 有 $P + N = \text{sum}$ 且 $P - N = target$
- 解得 $P = (\text{sum} + target)/2$
- 问题转化为：从数组中选一些数，和为 P，求方案数

四步法

- 状态：$dp[j]$ 表示凑出和 j 的方案数
- 转移：$dp[j] = dp[j] + dp[j - nums[i]]$
- 边界：$dp[0] = 1$
- 顺序：每个数只用一次，容量倒序

“计数型”背包：转移用加法，初始状态 $dp[0] = 1$

目标和：代码


```cpp
// dp[j] 表示凑出和 j 的方案数
int findTargetSumWays(vector<int>& nums, int target) {
    int sum = 0;
    for (int x : nums) sum += x;
    // P = (sum + target) / 2 必须是整数且非负
    if (sum < target || (sum + target) % 2 != 0) return 0;
    int P = (sum + target) / 2;

    vector<int> dp(P + 1, 0);
    dp[0] = 1;                                  // 凑出 0 有 1 种方案 *

    for (int x : nums) {
        // 每个数只用一次，容量倒序 *
        for (int j = P; j >= x; j--) {
            dp[j] += dp[j - x];
        }
    }
    return dp[P];
```

目标和：代码


```
19 }
```

计数型背包与最优型背包的代码结构完全相同，只是转移从 $\max / \min$ 变为加法

目录

1 背包问题概述

2 0/1 背包

3 完全背包

4 多重背包

5 分组背包

6 经典例题

目录

⑦ 总结

四类背包对比速查

类型	状态	转移	容量方向
0/1 背包	$dp[j]$ 最大价值	$\max(dp[j],\ dp[j-w]+v)$	倒序
完全背包	$dp[j]$ 最大价值	$\max(dp[j],\ dp[j-w]+v)$	正序
多重背包	二进制拆分后同上	拆成 0/1 处理	倒序
分组背包	$dp[j]$ 最大价值	每组内取 $\max$	组内倒序

四类背包的代码骨架完全一样，区别只在两个地方：容量方向、拆分方式

背包问题的心法

- 一切背包都是**选择问题**：对每个物品，选或不选
- 状态定义回答“已经考虑了什么”和“还剩多少容量”
- 转移方程回答“这一件物品怎么处理”
- 遍历顺序回答“同一物品能取几次”
- 一维优化的关键在**容量遍历方向**

背包 DP 的核心，不是记模板，而是理解“每个状态从哪来”

常见易错点

- 0/1 背包一维写成正序：变成完全背包，答案错误
- 完全背包一维写成倒序：每件只能用一次，答案偏小
- 多重背包二进制拆分时，忘了处理剩余部分
- 分组背包内层用正序：同一组可能被选多次
- 可行性 / 计数型背包，忘记初始化 $dp[0]$
- 转移方程中读错下标：$dp[i-1]$ 写成 $dp[i]$
- 容量为 0 时忘记单独处理，导致数组越界

下一步学什么

- 熟练写出四类背包的一维代码，能脱口而出遍历方向
- 能把新问题转化为背包：识别“物品、容量、目标”三要素
- 掌握三种常见变体：可行性、最少数量、方案计数
- 为后续学习更复杂的 DP 打基础：
  - 树形背包、依赖背包
  - 状态压缩背包、区间 DP
  - 单调队列优化多重背包

背包是 DP 的入门砖，也是走向复杂 DP 的必经路

附录：背包变体速查

变体	状态含义	转移方式
最大价值	容量 j 时的最大价值	max
最少数量	凑出 j 的最少物品数	min
可行性	能否凑出 j	布尔“或”
方案计数	凑出 j 的方案数	加法

识别题型的关键：问的是“最大 / 最少 / 能否 / 几种”

附录：复杂度对比

类型	时间复杂度	空间复杂度
0/1 背包（二维）	$O(nW)$	$O(nW)$
0/1 背包（一维）	$O(nW)$	$O(W)$
完全背包（一维）	$O(nW)$	$O(W)$
多重背包（二进制优化）	$O(W\sum \log c[i])$	$O(W)$
分组背包	$O(W\sum \lvert group_i \rvert)$	$O(W)$

一维优化是所有背包的标配，务必掌握

附录：一道题判断用哪种背包

- 每种物品最多一次 $\rightarrow$ 0/1 背包（倒序）
- 每种物品无限次 $\rightarrow$ 完全背包（正序）
- 每种物品有限 c 件 $\rightarrow$ 多重背包（二进制拆分）
- 每组只能选一个 $\rightarrow$ 分组背包（组内倒序）
- 只要“能否凑出” $\rightarrow$ 布尔型背包
- 要“最少物品数” $\rightarrow$ $\min$ 型背包
- 要“方案总数” $\rightarrow$ 加法型背包

附录：四种背包代码对照

```cpp
// 0/1 背包：倒序
for (int i = 0; i < n; i++)
    for (int j = W; j >= w[i]; j--)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

// 完全背包：正序
for (int i = 0; i < n; i++)
    for (int j = w[i]; j <= W; j++)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

// 多重背包：二进制拆分 + 0/1
for (int i = 0; i < n; i++) {
    int c = cnt[i];
    for (int k = 1; k <= c; k <<= 1) {
        c -= k;
        for (int j = W; j >= k * w[i]; j--)
            dp[j] = max(dp[j], dp[j - k*w[i]] + k*v[i]);
    }
```

附录：四种背包代码对照


```
19      if (c > 0)
20          for (int j = W; j >= c * w[i]; j--)
21              dp[j] = max(dp[j], dp[j - c*w[i]] + c*v[i]);
22  }
23
24  // 分组背包：组内倒序
25  for (int g = 0; g < n; g++)
26      for (int j = W; j >= 0; j--)
27          for (auto& item : groups[g])
28              if (j >= item.w)
29                  dp[j] = max(dp[j], dp[j - item.w] + item.v);
```